Symbolic Math Toolbox

1. Introduction

The **Symbolic Math Toolbox** in MATLAB is used for **analytical mathematics**. It allows users to work with exact mathematical expressions instead of decimal approximations. With this toolbox, students can perform symbolic algebra, solve equations, differentiate, integrate, compute limits, manipulate matrices symbolically, and work with high-precision arithmetic.

In normal MATLAB programming, values are often numerical, such as `3.14` or `5`. In symbolic mathematics, MATLAB can preserve exact mathematical objects such as:

$$\pi, \quad \sqrt{2}, \quad x^2 + 3x + 1$$

rather than immediately converting them to decimal form.

2. Creating Symbolic Variables

The most common command used to create symbolic variables is `syms`.

```
syms x y
f = x^2 + 3*x + 2
```

You can also create symbolic variables using `sym`.

```
x = sym("x");
y = sym("y");
```

Note: `syms` is a shorthand form of `sym`, but the two do not always behave exactly the same when assumptions are involved.

3. Symbolic Expressions

A symbolic expression is formed by combining symbolic variables with constants and mathematical operations.

```
syms x
f = x^3 - 4*x + 7
g = sin(x)^2 + cos(x)^2
```

Symbolic expressions are useful because they can be simplified, differentiated, integrated, substituted, and solved exactly.

4. Simplifying Expressions

The command `simplify` is used to reduce an expression to a simpler form.

```
simplify(g)
```

Example:

```
syms x
simplify(cos(x)^(-2) - tan(x)^2)
```

Other useful commands include:

```
expand((x-1)*(x+1))
factor(x^3 + 6*x^2 + 11*x + 6)
horner(x^5 + x^4 + x^3 + x^2 + x)
```

- `expand` rewrites expressions into expanded polynomial form.
- `factor` rewrites polynomials as products of factors.
- `horner` writes a polynomial in nested form for efficient evaluation.

5. Substitution with `subs`

The command `subs` is used to substitute variables in symbolic expressions with numbers, other variables, or other expressions.

```
syms x
f = 2*x^2 - 3*x + 1;
subs(f, x, 1/3)
```

Multiple substitutions:

```
syms a b c
subs(cos(a) + sin(b) - exp(2*c), [a b c], [pi/2 pi/4 -1])
```

Important: `subs` does not permanently change the original expression unless the result is assigned to a variable.

6. Solving Equations with `solve`

The command `solve` is used to obtain exact symbolic solutions.

```
syms x
solve(x + 1 == 2, x)
```

Quadratic equation example:

```
syms a b c x
eqn = a*x^2 + b*x + c == 0;
sol = solve(eqn, x)
```

System of equations:

```
syms x y
sol = solve([x + y == 5, x - y == 1], [x y])
```

Important: When forming equations for `solve`, use `==` and not `=`.

7. Numerical Solving with vpasolve

Sometimes a symbolic exact answer is difficult or impossible to express neatly. In that case, MATLAB can compute a numerical approximation using `vpasolve`.

```
syms x
vpasolve(sin(x) == 1/2, x)
```

Using an initial guess:

```
vpasolve(sin(x) == 1/2, x, 3)
```

Higher precision:

```
digits(50)
syms x
vpasolve(exp(x) == 3, x)
```

8. Assumptions on Variables

Assumptions help MATLAB determine the correct mathematical domain of a variable.

```
syms x positive
solve(x^2 + 5*x - 6 == 0, x)
```

Creating variables with assumptions using `sym`:

```
x = sym("x","real");
y = sym("y","positive");
```

To clear assumptions:

```
assume(x,"clear")
```

Assumptions can affect simplification and the solutions returned by symbolic computations.

9. Differentiation

The command `diff` is used to differentiate symbolic expressions.

```
syms x
f = x^3 + 2*x^2 - 5*x + 1;
df = diff(f, x)
d2f = diff(f, x, 2)
```

For symbolic functions:

```
syms f(x)
f(x) = x^2*sin(x);
diff(f, x)
```

10. Integration

The command `int` is used for symbolic integration.

10.1 Indefinite Integral

```
syms x
int(x^2, x)
```

10.2 Definite Integral

```
syms x
int(x^2, x, 0, 2)
```

Another example:

```
syms x
f = exp(-x^2);
int(f, x)
```

11. Limits and Taylor Series

11.1 Limits

```
syms x
limit(sin(x)/x, x, 0)
```

11.2 Taylor Series Expansion

```
syms x
taylor(exp(x), x)
taylor(sin(x), x, pi/4)
```

The Taylor series is useful for approximating complicated functions with polynomials.

12. Symbolic Functions

Symbolic functions explicitly depend on variables and are useful in calculus.

```
syms f(x)
f(x) = x^2 + 1;
f(3)
diff(f, x)
int(f, x)
```

13. Symbolic Matrices

The toolbox also allows symbolic matrix operations.

```
syms x y
A = [x 1; y 2];
det(A)
inv(A)
eig(A)
```

This is useful in linear algebra, control systems, and engineering mathematics.

14. Variable-Precision Arithmetic with `vpa`

The command `vpa` computes decimal approximations to many digits.

```
vpa(pi, 50)
vpa(sqrt(sym(2)), 40)
```

This is useful when an exact symbolic result exists but a high-precision decimal approximation is required.

15. Converting Symbolic Expressions to MATLAB Functions

Use `matlabFunction` to convert symbolic expressions into ordinary MATLAB functions.

```
syms x
f = x^2 + 1;
g = matlabFunction(f);
g(3)
```

To write the function to a file:

```
matlabFunction(f, "File", "myFunction")
```

16. Worked Classroom Examples

16.1 Example 1: Simplify an Identity

```
syms x
simplify(sin(x)^2 + cos(x)^2)
```

16.2 Example 2: Solve a Quadratic Equation

```
syms x
solve(x^2 - 5*x + 6 == 0, x)
```

16.3 Example 3: Differentiate and Integrate

```
syms x
f = x^3 - 2*x + 4;
diff(f, x)
int(f, x)
```

16.4 Example 4: Evaluate at a Point

```
syms x
f = x^2 + 3*x + 1;
subs(f, x, 2)
```

16.5 Example 5: Numerical Solution

```
syms x
vpasolve(cos(x) == x, x)
```

17. Common Mistakes Students Make

1. Forgetting to declare symbolic variables using `syms`.
2. Using `=` instead of `==` in equations.
3. Expecting `subs` to permanently change an expression without reassignment.
4. Ignoring assumptions on variables.
5. Confusing exact symbolic results with decimal approximations.

18. Summary

The **Symbolic Math Toolbox** enables MATLAB to work with mathematics in exact symbolic form instead of only numeric approximations. The most important beginner-to-intermediate commands are:

`syms` `simplify` `expand` `factor` `subs` `solve` `vpasolve`

`diff` `int` `limit` `taylor` `vpa` `matlabFunction`

Mastering these commands gives students a strong foundation in symbolic computation within MATLAB.

19. Practice Exercises

1. Create symbolic variables x and y, then form the expression:

$$x^2 + 4x + y$$

2. Simplify the expression:

$$\sin^2(x) + \cos^2(x)$$

3. Expand:

$$(x + 2)(x - 3)$$

4. Factor:

$$x^2 - 5x + 6$$

5. Solve the equation:

$$x^2 - 9 = 0$$

6. Find the derivative of:

$$f(x) = x^4 - 3x^2 + 2$$

7. Find the integral of:

$$f(x) = 3x^2 + 2x + 1$$

8. Evaluate the expression $x^2 + 3x + 1$ at $x = 4$.

9. Find the limit:

$$\lim_{x \to 0} \frac{\sin x}{x}$$

10. Use `vpa` to compute π correct to 30 decimal places.